F1 Podium Finish Prediction Using Race Result Data

CAI 4105 - Introduction to Machine Learning

Professor Dr. Mustafa Ocal

Trevor Werner

Introduction

My project is based on my interest in Formula 1 racing. I will be using data from races spanning from 2019-2024. After the recent race here in Miami, I was curious about the variety of factors that contribute to a successful team season and how historical data could be leveraged to potentially predict future race results. While some more advanced approaches focus on data specific to a vehicle's telemetry, environmental conditions, or more granular points of information, I will be focusing on analyzing more general race data to compare and contrast various teams' historical performances. The goal of this project will be to predict a podium vs. non-podium finish for a future race provided similar feature data.

Related Work

The GitHub notebook "Formula 1 Race Results 2019–2024" provides a practical example of exploratory data analysis using race result data. It demonstrates how F1 data can be loaded, inspected, cleaned, and visualized to better understand patterns in driver and team performance. Although it is not a formal research paper, it is useful for this project because it works with a similar type of race-level dataset and highlights the importance of preprocessing before applying machine learning methods.

The "Formula 1 Race Results" repository provided a useful template for my project and included an exploratory analysis section that focused on top drivers, top teams, ranking drivers by win count, podium finishes, fastest laps, points by season, along with a variety of other vectors of comparison. The feature engineering section employed one-hot encoding, scikit-learn's simple imputer, along with other scikit-learn built-in module methods. The training and test split was also 80/20, and the models employed for prediction were Logistic Regression, Random Forest, Gradient Boosting, and Extra Trees. The target for these models was also "Podium Finishes"; However, this project did not include 2nd place finishes.

Performance:

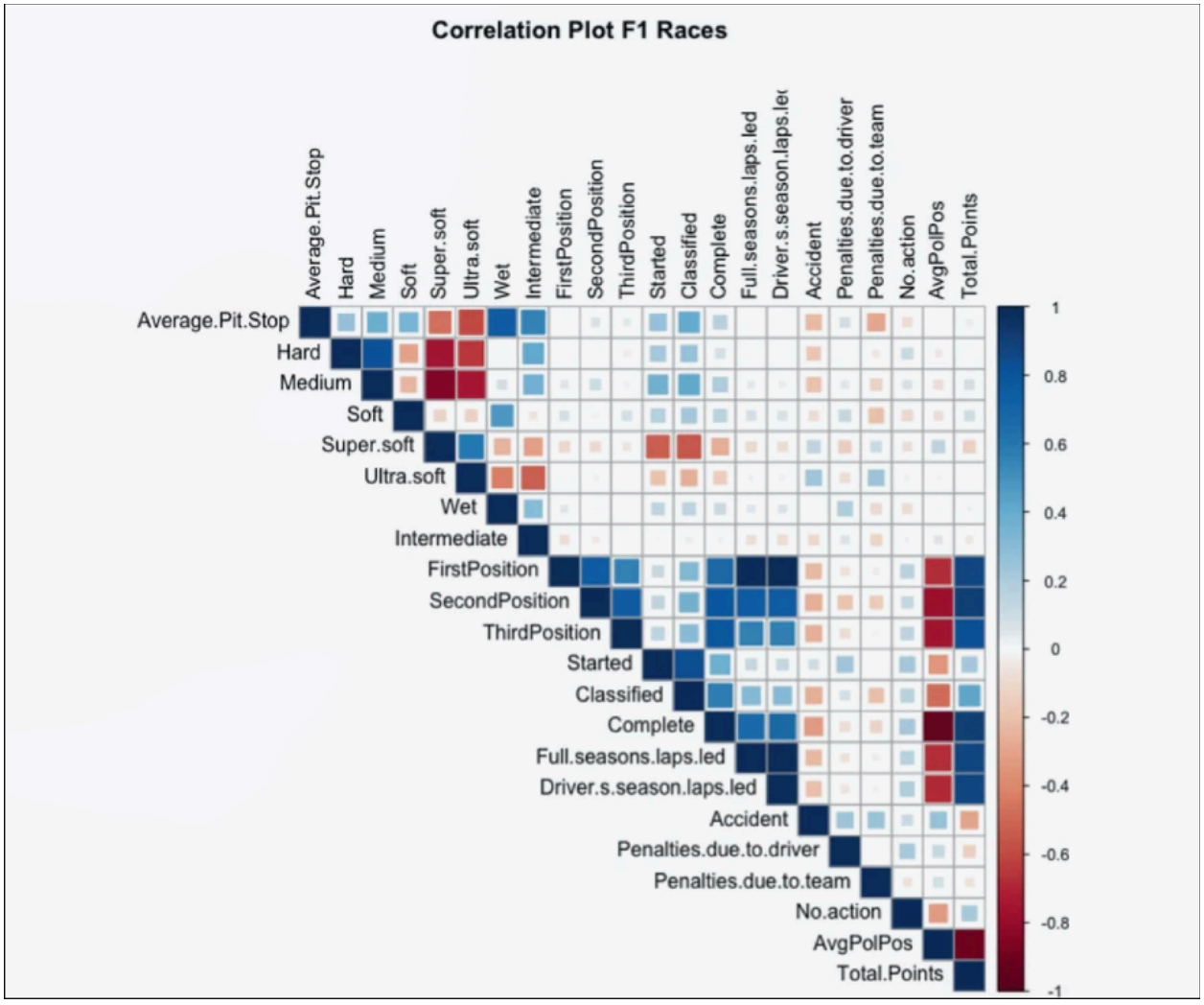
Model Comparison

	Model	Accuracy	Precision	Recall	F1	ROC_AUC
0	Logistic Regression	0.912109	0.705128	0.714286	0.709677	0.944380
2	Gradient Boosting	0.910156	0.701299	0.701299	0.701299	0.937035
1	Random Forest	0.898438	0.676056	0.623377	0.648649	0.929482
3	Extra Trees	0.892578	0.644737	0.636364	0.640523	0.917734

The best ROC AUC (Receiver Operating Characteristic - Area Under the Curve) score for this project was 94.4% from the Logistic Regression model trained with max iterations set to 5000.

Patil et al. (2023) conducted a data-driven analysis of Formula 1 race outcomes using historical racing data. Their work is relevant because it treats Formula 1 as a prediction and analytics problem rather than simply a sports summary problem. Race outcome prediction is challenging because results are influenced by many interacting variables, including driver ability, team performance, race conditions, and starting position. This supports my decision to simplify the task into a binary podium classification problem instead of attempting to predict exact finishing order.

In the Springer Nature research article “A Data-Driven Analysis of Formula 1 Car Races Outcome”, data from F1 races between 2015 and 2019 are explored and a Principal Component Analysis (PCA) is performed to identify the most impactful attributes out of a larger space of 21 input variables The article breaks their research into four main categories for strategy that include: preparation, practice, qualifying and racing.



As the article mentions, based on the results of the correlation plot (above), pole position is highly correlated with race outcome. The article notes “We observe that the average pole position is strongly negatively correlated with the first, second and third position.”, and continues with an observation that team penalties “appear to be related to the usage of soft tyres and hyper soft tyres”.

The Catapult article discusses how data analysis is used in modern Formula 1 to improve performance, strategy, and decision-making. While the article is industry-focused rather than academic, it provides useful context for why F1 is an appropriate domain for machine learning. It shows that race teams rely heavily on data to understand performance and make decisions. My project applies this idea on a smaller scale by using publicly available race result data to

predict podium finishes.

In this analysis, the real-world integration of Catapult's solutions "Racewatch" and "Circuit Manager" is discussed in depth. The main advantages covered for integration of these modern tools are in the categories of vehicle performance optimization through telemetry data, predictive analytics for improving strategy, as well as driver analytics (how do drivers behave under different environmental conditions)

The article continues to discuss how the sport is evolving through the leveraging of big-data analytics, AI and machine learning integration, Virtual and Augmented Reality technologies, and Internet of Things (IoT) telemetry data. Hardware provides more "granular" data on areas such as tire wear, aerodynamics, and engine output, while AI and Machine Learning facilitate strategy improvements.

Data

The dataset provides a variety of numerical and categorical features across **15** columns with a total of **2559** unique rows.

Attributes

Attribute	Type	Description	Missing	Role in project
`Track`	string	Grand Prix circuit name (34 circuits)	0	Model feature (one-hot)
`Position`	string	Finishing position (1–20, `NC`, or `DQ`)	0	Used to derive `podium_finish`
`No`	integer	Driver/car number (1–99)	0	Not used
`Driver`	string	Driver name (36 drivers)	0	Model feature (one-hot)
`Team`	string	Constructor/team name (22 teams)	0	Model feature (one-hot)
`Starting Grid`	float	Starting grid slot (1–20)	1	Model feature
`Laps`	integer	Laps completed (0–87)	0	Not used
`Time/Retired`	string	Race finish time or retirement reason	760	Not used
`Points`	float	Championship points awarded (0–26)	0	Not used
`+1 Pt`	string	Extra point for fastest lap (`Yes` / `No`)	1,679	Not used
`Fastest Lap`	string	Lap time when driver set fastest lap	970	Not used
`season`	integer	F1 season year (2019–2024)	0	Train/validation split only
`Set Fastest Lap`	string	Whether driver set fastest lap (`Yes` / `No`)	1,640	Not used
`Fastest Lap Time`	string	Fastest lap time in the race	1,669	Not used
`Total Time/Gap/ Retirement`	string	Gap to winner or retirement status	1,799	Not used
`podium_finish`	integer	Binary target: 1 = podium (1st–3rd), 0 = otherwise	—	Target variable (derived)

My target variable is a binary classification of a podium finish (1st, 2nd, or 3rd place) vs. a non-podium finish and I've sourced my data from Kaggle, and the dataset was downloaded in csv format for ingestion.

Initial inspection of the data indicates zero duplicates and a significant number of missing values

for a small group of the 15 feature columns (ex., the “Fastest Lap” column contains **970** empty values)

I excluded the racer number column (index) from my model training, as it is primarily a unique identifier and not useful for analysis and dropped the “+1” column from my model training to prevent data leakage from the training into the test set.

Data Leakage is a potential concern for some of the features whose values are conditional on the target/dependent variable (e.g., the “+1” column is a bonus for top performers). Columns such as “Total Time/Gap/Retirement” and “Time/Retired” need to be removed or adjusted due to their conditional linkage to the outcome variable (podium finish). Because the target variable is derived from the final race position, I will avoid all race outcome fields.

Implementation

The libraries used for this project included: pandas and numpy (data analysis), matplotlib and seaborn (visualizations), along with scikit-learn and xgboost (machine learning)

The pipeline stages were broken into 7 main phases:

1. Ingestion
2. Exploratory Data Analysis (EDA)
3. Preprocessing (handling missing values, converting data types, etc.)
4. Feature Scaling (standard scaling applied for value normalization)
5. Model training
6. Hyperparameter tuning (XGBoost was the model selected to optimize)
7. Evaluation (Confusion Matrix, ROC-AUC score)

Performance was measured primarily through comparison between precision (testing proportion of true positives), recall (proportion of actual positives correctly identified) f1-score (harmonic mean of precision and recall) and accuracy (combining the measurement of both successful positives and negatives).

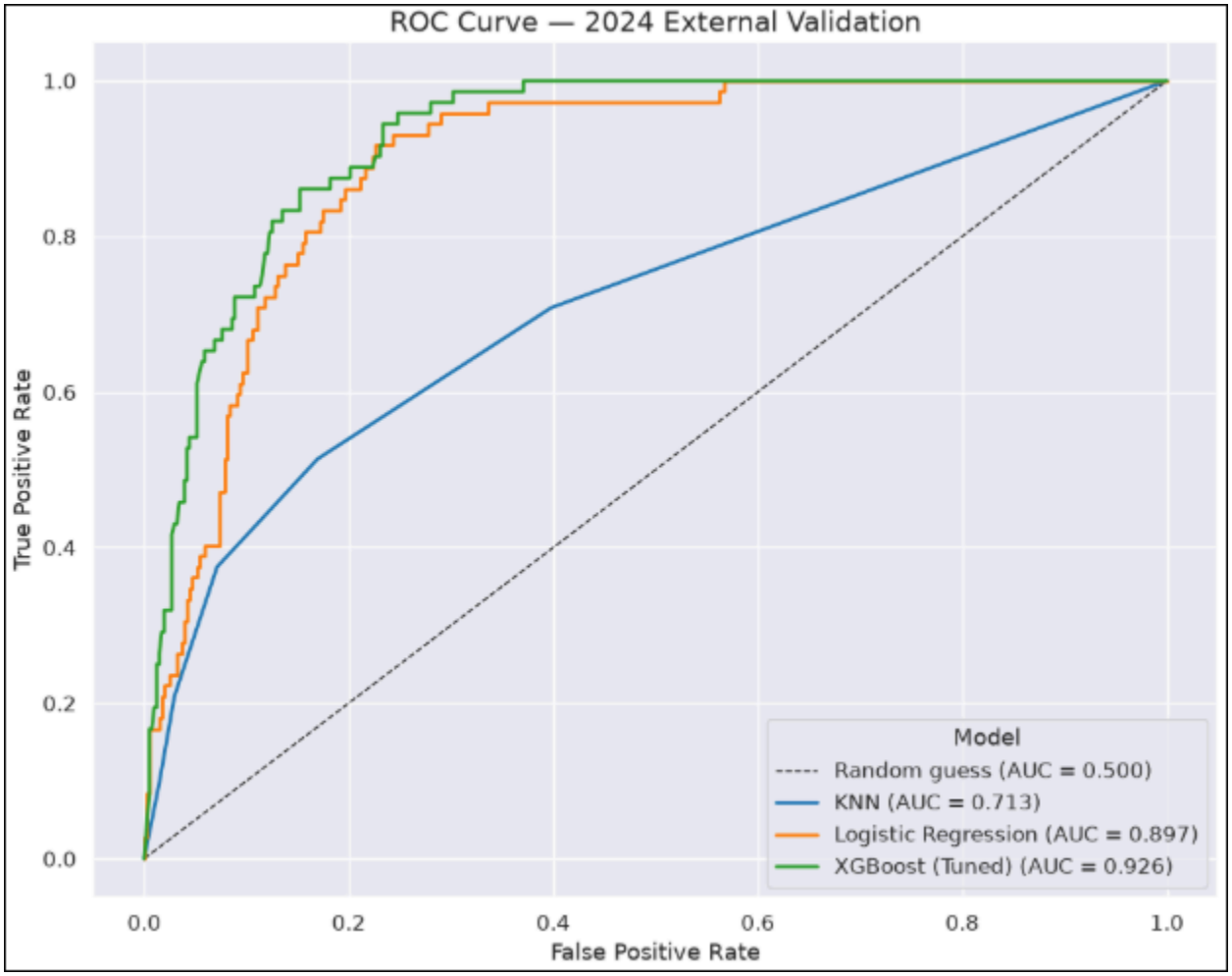
Modeling began with K-Nearest Neighbors (KNN) classifier initially and was compared against results from Logistic Regression and an XGBoost implementation.
I used a training/test split of 80% segregated for training and the remaining 20% for testing, and applied this consistently for each model.

Results

After comparing 3 different classification models: K-Nearest Neighbors, Logistic Regression, and XGBoost, it was clear that the XGBoost model was the winner in terms of overall performance, and these results are illustrated in the following ROC-AUC plot:
Model Comparison

	Model	Accuracy	Precision	Recall	F1
0	KNN (Test)	0.894	0.688	0.532	0.600
1	KNN (2024)	0.846	0.482	0.375	0.422
2	Logistic Regression (Test)	0.904	0.696	0.629	0.661
3	Logistic Regression (2024)	0.854	0.518	0.403	0.453
4	XGBoost (Tuned) (Test)	0.897	0.686	0.565	0.619
5	XGBoost (Tuned) (2024)	0.887	0.650	0.542	0.591

ROC-AUC Plot



The XGBoost model was tuned using the GridSearchCV library, which identified the optimal parameters for n_estimators, max_depth, learning_rate, subsample, and column sample bytree. The tuned model produced the strongest 2024 validation performance, with an accuracy of **0.887**, F1-score of 0.591 and ROC-AUC of 0.926

Discussion

The overall performance was better than expected, with a modest runtime for training and output (under 2 minutes). The dataset features were informative but ultimately less than half were usable for model training. My biggest concern prior to training was to prevent data leakage from outcome-informing being utilized in the training of each model. Columns such as

“Fastest Lap”, “Set Fastest Lap” and “Fastest Lap Time” were too closely interrelated and had disproportionate influence on the target variable (podium finish).

Future Work

Possible areas to expand this project include incorporating race-day data, including variables (features) such as tire choice, weather conditions, penalties by team (if assessed), pit strategy, and the mechanical state of team vehicles.

The final predictive input features were: Starting Grid (position), Driver (name), Team, Track (location), season, and podium_finish (target separated for prediction). The season column was used only to create the 2019-2023 training period separation from the 2024 validation holdout. Future versions of this project could potentially include any of the race-day variables discussed in the Catapult article.

References

1. Patil, A., Jain, N., Rahul Agrahari, Murhaf Hossari, Orlandi, F., & Dev, S. (2023). A Data-Driven Analysis of Formula 1 Car Races Outcome [Review of *A Data-Driven Analysis of Formula 1 Car Races Outcome*]. *Springer Nature*, 134–146. https://doi.org/10.1007/978-3-031-26438-2_11

2. Ambler, W. (2024, February 13). F1 Data Analysis & Technology: How Data is Transforming Race Performance. Catapult. https://www.catapult.com/blog/f1-data-analysis-transforming-performance

3. Sonawane, L. (2026). formula-1-race-results-2019-24-94.ipynb (Commit 79b74be) [Jupyter Notebook]. GitHub. https://github.com/lalitssonawane/100-days-of-data-science-projects/blob/79b74bef5684f2bec33d6ee6c8d4e46c9ca3f2a5/formula-1-race-results-2019-24-94.ipynb